

GlideQuery: chain rules, Stream, and Optional

This note explains why a GlideQuery chain stops, how to store a query and add to it, and which methods belong to Stream versus Optional. Names follow the official API: **orderBy** / **orderByDesc**, and **deleteMultiple**. **orderByAsc** and a GlideQuery **delete()** are not in the documented method list. In a scoped app the class is **global.GlideQuery**.

1. The three objects

GlideQuery is not one object with every method on it. A script holds one of three things, and a method exists only on the object you are holding.

Object	How you get it	What you can call
GlideQuery	new GlideQuery(table), or any intermediate call	where, orWhere, whereNull, whereNotNull, orWhereNull, orWhereNotNull, orderBy, orderByDesc, limit, groupBy, aggregate, having, withAcls, disableWorkflow, forceUpdate, disableAutoSysFields
Stream	select()	map, filter, limit, flatMap (still a Stream). forEach, reduce, toArray, find (these end the Stream). find returns an Optional.
Optional	selectOne(), get(), getBy(), insert, update, insertOrUpdate, Stream.find()	isPresent, isEmpty, ifPresent, map (still an Optional). get and orElse unwrap it.

Intermediate GlideQuery methods return a new GlideQuery, so they chain. A terminal GlideQuery method leaves the builder. select returns a Stream. selectOne, get, getBy, insert, and update return an Optional. count returns a number. updateMultiple returns an object with rowCount. deleteMultiple returns nothing. parse is static: GlideQuery.parse(table, encodedQuery). It is not a method on an instance.

2. Basics

Compared with GlideRecord, you do not call query() or next(). Each builder call returns a new object instead of mutating one record pointer. Field values come back as plain JavaScript values, so a string comparison with === works. A typo in a field name throws NiceError and runs nothing. The same typo on GlideRecord is ignored, and a following deleteMultiple can match every row.

What GlideQuery is — and what it is not

A server-side fluent API for reading and writing records. You build a query, then one terminal call runs it. In a scoped app the class is **global.GlideQuery**.

COMPARED WITH GLIDERECORD

- GlideRecord mutates one object: **addQuery()**, then **query()**, then **next()**.
- GlideQuery returns a new object from each builder call. You do not call **query()** or **next()**.
- A bad field name fails fast. GlideRecord often ignores it and matches every row.
- Results are plain objects, not GlideElement. **===** works on strings.

THREE OBJECTS, NOT ONE

- **GlideQuery** — the builder. `where` `orderBy` `limit`
- **Stream** — many rows, from **select()**. `forEach` `map`
`toArray`
- **Optional** — one row or none, from **selectOne()** / **get()**.
`orElse` `isPresent`

A method exists only on the object you are holding. That is the whole chaining rule.

THE SHAPE OF A READ

```
new GlideQuery('incident')
  .where('active', true)
  .orderBy('number')
  .limit(20)
  .select('number')
  .forEach(function (row) {
    gs.info(row.number);
  });
```

Builders first. **select** hands you a Stream. **forEach** is what actually pulls rows.

STORE THE BUILDER, THEN ADD TO IT

```
var open = new GlideQuery('incident')
  .where('active', true);

var p1 = open.where('priority', 1).count();
var p2 = open.where('priority', 2).select('number');

// p2 is a Stream — no where()
p2.where('priority', 1);
```

Each **where** returns a new GlideQuery. **open** is not changed. **count()** returns a number. **select()** returns a Stream.

FAIL-FAST, BEFORE ANY WRITE

- Unknown field: **activated** instead of **active** throws. It does not match every row.
- Bad choice and wrong type throw the same way.
- **update()** needs a primary-key **where**, usually **sys_id**.
- Sets use **updateMultiple()**. Deletes use **deleteMultiple()**. There is no single-record delete.
- Run the same **where** chain through **count()** before a bulk write.

Figure 1. What GlideQuery is, how a stored builder is extended, and the fail-fast write rules.

A normal read

```
new GlideQuery('incident')
  .where('active', true)
  .orderBy('number')
  .limit(20)
  .select('number', 'short_description')
  .forEach(function (row) {
    gs.info(row.number + ' ' + row.short_description);
  });
```

where, orderBy, and limit are still a GlideQuery. select is the handoff: it returns a Stream, and sys_id is always included even if you did not ask for it. forEach is a Stream method. That is the call that pulls rows. select itself is lazy — mapping and filtering on the Stream do no work until forEach, reduce, toArray, or find.

3. Why select().where() fails

where is a method on GlideQuery. select does not return a GlideQuery, so the next call is not looking up where on the builder. It is looking up where on a Stream, and Stream has no where. The failure is a missing method, not a rejected filter.

```
new GlideQuery('incident') // GlideQuery
  .where('active', true) // GlideQuery
  .select('number') // Stream — builder is over
```

```
.where('priority', 1); // Stream has no where()
```

Terminal means the GlideQuery chain is over. It does not mean the database already ran. You can still map and filter the Stream. You cannot go back and add a query builder method. If the filter belongs in the query, put where before select. Stream.filter after select filters rows already headed for script, which is a different operation.

4. Storing a query and adding to it

You store the builder. Intermediate calls return a new GlideQuery, so a variable can be extended. The graphic that says a terminal returns nothing GlideQuery-shaped is only about that last call. Calls before it are how the variable stays usable.

```
var open = new GlideQuery('incident').where('active', true);

// each where returns a new GlideQuery; open is unchanged
var p1 = open.where('priority', 1).count(); // number
var p2 = open.where('priority', 2).select('number'); // Stream

p2.where('priority', 1); // invalid: p2 is a Stream
```

Reassigning also works, because the right-hand side is still a GlideQuery: incidents = incidents.where('priority', 1).orderBy('number'). The terminal stays last. count and select are not stored if you still need to add conditions.

5. Chain rules

Green methods return a GlideQuery. Amber methods execute and end the builder. Blue methods belong to Stream or Optional. Red rows are calls that fail because the receiver is the wrong object, or because the write is unbounded.

SERVICENOW · SERVER API

GlideQuery chain rules — validated

Every method is intermediate or terminal. Intermediate returns a new GlideQuery. Terminal executes and the chain is over. Scoped apps must use **global.GlideQuery**.

Official classification
Zurich / current docs
orderBy · deleteMultiple

● THE ONE RULE

new GlideQuery → where · orderBy · limit →

select / count / update

Builders first. A terminal returns a Stream, Optional, number, GlideRecord, or {rowCount}. Nothing GlideQuery-shaped comes back.

● INTERMEDIATE · KEEP CHAINING

Returns GlideQuery. Safe to chain.

● TERMINAL · CHAIN ENDS

Executes. Next call is Stream, Optional, or a value — not GlideQuery.

● INTERMEDIATE METHODS

where orWhere whereNull whereNotNull orWhereNull orWhereNotNull orderBy

orderByDesc limit groupBy aggregate having withAcls disableWorkflow

forceUpdate disableAutoSysFields

where(field, 'IN', [...]) is the membership form. No whereIn in the official list. limit() here runs before the query — use this, not Stream.limit().

● TERMINAL METHODS

select selectOne get getBy count sum max min avg insert

insertOrUpdate update updateMultiple deleteMultiple toGlideRecord

select → Stream (sys_id always included). selectOne / get / getBy / insert / update → Optional. count → number. updateMultiple → {rowCount}. deleteMultiple → nothing. parse() is static, neither.

● CHAINS THAT FAIL

INVALID	WHY
select().where()	select returned a Stream
count().select()	count returned a number
select().orderBy()	builders must come first
insert().where()	no row set to filter
update() without sys_id where	single-record update needs the primary key
new GlideQuery().forEach()	forEach is Stream-only
get().get()	second get unwraps an Optional
instance.parse()	parse is static: GlideQuery.parse(table, eq)

● AFTER THE TERMINAL

Not GlideQuery. Do not chain query builders onto these.

Stream.forEach toArray map filter reduce find flatMap

limit

Optional.get orElse isPresent isEmpty ifPresent

Optional.get() throws if empty. Prefer isPresent() when absence is normal. Stream.limit() after select works but reads then trims — mark it bad.

● FAIL-FAST VALIDATION · EXECUTES NOTHING

Unknown field

where('activated', '!=', true) → NiceError, not a full-table delete

Invalid choice

where('approval', 'donotexist') → lists allowed values

Wrong type

where('state', '8') on an integer field → type mismatch

insert / update also check field, type, and choice. Before a bulk write, run the same where() chain through count().

● NAMES, SETTLED

• orderBy / orderByDesc — official. orderByAsc is not in the API.

• deleteMultiple() — official. No single-record delete. del() is an alias. delete() is not documented.

• update() needs a primary-key where. Sets use updateMultiple().

• Bare updateMultiple / deleteMultiple with no where is unsupported. Docs require preceding where clauses; the exact error string is not quoted.

Sources: ServiceNow GlideQuery API (Zurich / current docs), GlideQuery blog (data integrity, no single delete), script-include alias del = deleteMultiple.

Figure 2. Official intermediate and terminal methods, invalid chains, and fail-fast checks.

Invalid chains

Call	Why it fails
<code>select().where() / select().orderBy()</code>	<code>select</code> returned a <code>Stream</code> . Builders must come first.
<code>count().select()</code>	<code>count</code> returned a number.
<code>insert().where()</code>	An insert has no row set to filter.
<code>update()</code> without a primary-key where	Single-record update requires the primary key, usually <code>sys_id</code> . Use <code>updateMultiple</code> for a set.
<code>new GlideQuery(table).forEach(fn)</code>	<code>forEach</code> is <code>Stream</code> -only. <code>select</code> first.
<code>get().get()</code>	The first <code>get</code> is <code>GlideQuery.get(sys_id)</code> . The second is <code>Optional.get</code> , and it throws if empty.
<code>instance.parse(...)</code>	<code>parse</code> is static: <code>GlideQuery.parse(table, encodedQuery)</code> .
<code>selectOne().forEach(fn)</code>	<code>selectOne</code> returned an <code>Optional</code> . <code>Optional</code> has no <code>forEach</code> .
<code>orElse(v).isPresent()</code>	<code>orElse</code> already unwrapped the <code>Optional</code> . The result is a plain value.

6. When to use each object

Pick the object from the shape of the answer, not from habit. `GlideQuery` is the question. `Stream` is a list of answers. `Optional` is one answer that might be missing. `count`, `updateMultiple`, and `deleteMultiple` are neither: they return a number, a row count, or nothing.

Use	When	Why this one
<code>GlideQuery</code>	You are still deciding which rows. You may add a <code>where</code> later, or run the same filter as a <code>count</code> and as a <code>select</code> .	<code>where</code> and <code>orderBy</code> exist only here. Storing this is what lets you extend the query. A terminal throws that away.
<code>Stream</code> , via <code>select()</code>	The answer is a set: zero, one, or many rows, and you will loop, <code>map</code> , or <code>collect</code> them.	A set does not fit in an <code>Optional</code> . <code>select</code> is lazy, so <code>map</code> and <code>filter</code> cost nothing until <code>forEach</code> , <code>toArray</code> , or <code>reduce</code> .
<code>Optional</code> , via <code>selectOne()</code>	A filter should match one row, and no row is a normal result.	<code>selectOne</code> always returns an <code>Optional</code> , empty or not. You handle absence with <code>orElse</code> or <code>isPresent</code> instead of a null check on a <code>GlideRecord</code> .
<code>Optional</code> , via <code>get(sys_id)</code>	You already have the primary key.	<code>get</code> is the lookup. It still returns an <code>Optional</code> , because that <code>sys_id</code> may not exist.
<code>Optional</code> , via <code>getBy({...})</code>	You have field values, not a <code>sys_id</code> , and you only want one match.	<code>getBy</code> returns one row even if several match. Do not use it if you need every match — that is <code>select</code> .
<code>Optional.get()</code>	A missing row is a bug, not a case to handle.	<code>get</code> unwraps the <code>Optional</code> and throws if it is empty. That is the wrong call when absence is normal.

`Stream` versus `Optional` is the same choice as a list versus a single value. If you would write a `while (gr.next())` loop, use `select` and a `Stream`. If you would write `if (gr.get(sys_id))`, use `get` or `selectOne` and an `Optional`. Using `select` for one row forces a loop around a single object. Using `selectOne` for many rows silently keeps only the first.

Stream — a set of incidents

Use this when the caller needs every open P1, not one of them. `limit` goes on the `GlideQuery` so the database stops at 20. `forEach` is what reads the rows.

```
// why Stream: the result is a list, and zero rows is fine – forEach just does not run
var openP1 = new GlideQuery('incident')
.where('active', true)
.where('priority', 1)
.orderBy('number')
.limit(20);

openP1.select('number', 'short_description')
.forEach(function (row) {
gs.info(row.number + ' ' + row.short_description);
});

// same builder, different terminal – a count, not a Stream
gs.info('open P1 count ' + openP1.count());
```

Optional — one user, who might not exist

Use this when the caller asked for one user by email. `orElse` covers the normal miss. `get()` is only there if a missing user should stop the script.

```
// why Optional: one row or none. orElse supplies the default without a throw
var user = new GlideQuery('sys_user')
.where('email', 'abel.tuter@example.com')
.selectOne('first_name', 'email')
.orElse({ first_name: 'Nobody', email: '' });
gs.info(user.first_name);

// same lookup, absence is a bug – get throws on an empty Optional
var mustExist = new GlideQuery('sys_user')
.get('5137153cc611227c000bbd1bd8cd2005', ['first_name'])
.get();
gs.info(mustExist.first_name);
```

7. Every syntax that produces a Stream or an Optional

`select()` is the only `GlideQuery` method that creates a `Stream`. Nothing else on the builder does: not `selectOne`, not `get`, not `count`, not a `where`. Once you have that `Stream`, `map`, `filter`, `limit`, `flatMap`, and `chunk` return another `Stream`. They do not create the first one. `find` is the bridge out: `Stream` in, `Optional` out.

An `Optional` has more than one entrance. `selectOne`, `get`, and `getBy` wrap one row. `insert`, `update`, and `insertOrUpdate` wrap the written row. `sum`, `max`, `min`, and `avg` wrap one number, which is why `orElse(0)` is legal on them. `Stream.find` wraps the first matching row. `Optional.map` wraps whatever the function returns. `count` is not on this list: it returns a number. `updateMultiple` returns `{rowCount}`. `deleteMultiple` returns nothing.

Call	You are holding	Returns	Syntax
<code>select('f1', 'f2')</code>	<code>GlideQuery</code>	<code>Stream</code>	The only <code>GlideQuery</code> call that creates a <code>Stream</code> . <code>sys_id</code> is always included. No fields means every field.
<code>map / filter / limit</code>	<code>Stream</code>	<code>Stream</code>	Keep the <code>Stream</code> . Lazy until a <code>Stream</code> terminal. <code>limit</code> here trims after the query.

Call	You are holding	Returns	Syntax
flatMap(fn)	Stream	Stream	fn must return a Stream, usually another select(). flatMap flattens it. map would nest Stream inside Stream.
chunk(n)	Stream	Stream	Stream of arrays of n items. Still a Stream, so flatMap or forEach can follow.
forEach / reduce / toArray(n)	Stream	nothing / value / array	These end the Stream and run the read. Nothing Stream-shaped comes back.
find(fn)	Stream	Optional	First match, or empty. Optional methods are legal after this, Stream methods are not.
selectOne('f1')	GlideQuery	Optional	One row matching the where, or empty. Several matches: you get one, not a Stream.
get(sys_id, fields)	GlideQuery	Optional	Primary-key lookup. Empty if that sys_id is missing.
getBy({...}, fields)	GlideQuery	Optional	One row by field values. One even if many match. Every match is select, not getBy.
insert / update / insertOrUpdate	GlideQuery	Optional	The written row, or empty if the write did not return one. update still needs a primary-key where.
sum / max / min / avg	GlideQuery	Optional	One number, or empty if nothing matched. orElse(0) is the usual unwrap.
Optional.map(fn)	Optional	Optional	Another Optional. Chain it before orElse or get. It does not make a Stream.
isPresent / isEmpty	Optional	boolean	The test. They do not unwrap.
get() / orElse(v)	Optional	the row or value	Unwrap. get throws if empty. orElse returns the default. Neither returns an Optional.

Paths that stay a Stream

```
// only select() creates the Stream
var numbers = new GlideQuery('incident')
  .where('active', true)
  .select('number') // Stream
  .filter(function (row) { return row.number; }) // Stream
  .map(function (row) { return row.number; }) // Stream
  .limit(10); // Stream – trims after the read
numbers.forEach(gs.info); // ends it

// flatMap: inner select() makes a Stream, flatMap keeps one Stream
new GlideQuery('sys_user')
  .whereNotNull('first_name')
  .select('first_name')
  .flatMap(function (user) {
    return new GlideQuery('incident')
      .where('caller_id', user.sys_id)
      .select('number');
  })
  .forEach(function (inc) { gs.info(inc.number); });
```

A where after any of these is still invalid. The Stream was created by select, and where is not a Stream method. Put the filter on the GlideQuery, or use filter on the Stream if you mean an in-script check.

Paths that produce an Optional

```
// one row from a filter
var byEmail = new GlideQuery('sys_user')
.where('email', 'abel.tuter@example.com')
.selectOne('first_name')
.orElse({ first_name: 'Nobody' });

// one row from a primary key
var byId = new GlideQuery('sys_user')
.get('5137153cc611227c000bbd1bd8cd2005', ['first_name'])
.orElse(null);

// one number – sum returns an Optional, not a Stream and not a raw number
var faults = new GlideQuery('cldb_ci').sum('fault_count').orElse(0);

// Stream.find is the other entrance: select made the Stream, find makes the Optional
new GlideQuery('incident')
.where('active', true)
.select('number', 'short_description')
.find(function (row) { return row.short_description; })
.ifPresent(function (row) { gs.info(row.number); });

// written row
var created = new GlideQuery('incident')
.insert({ short_description: 'opened from script' })
.orElse(null);
```

selectOne on a set does not give you a Stream of the rest. It gives you one Optional. getBy does the same. If you need every match, the call is select, and the object is a Stream.

8. Stream versus Optional syntax

Use select when you expect many rows, including zero. Use selectOne, get, or getBy when you expect one row that might be missing. get(sys_id) looks up the primary key. getBy({field: value}) returns one match even if several rows qualify. insert, update, and insertOrUpdate also return an Optional of the written row.

Stream vs Optional — valid and invalid

`select()` returns a Stream (many rows). `selectOne()`, `get()`, and `getBy()` return an Optional (one row, or none). Neither has `where()`.

STREAM — FROM SELECT()

Still a Stream, so you can chain them: `map` `filter` `limit` `flatMap`

These end the Stream: `forEach` `reduce` `toArray` `find` — `find` returns an Optional.

```
new GlideQuery('incident')
  .where('active', true)
  .select('number')
  .filter(function (row) { return row.number; })
  .map(function (row) { return row.number; })
  .forEach(gs.info);

var rows = new GlideQuery('incident')
  .select('number').toArray(50);

.select('number').where('active', true);
.select('number').orderBy('number');
.forEach(gs.info).map(fn);
new GlideQuery('incident').filter(fn);
```

`map/filter/limit` are lazy. Rows are read when `forEach`, `reduce`, `toArray`, or `find` runs. `Stream.limit` trims after the query. `GlideQuery.limit`, before `select`, is the one that belongs in the query.

OPTIONAL — FROM SELECTONE / GET / GETBY

`isPresent` `isEmpty` return a boolean. `get` returns the row or throws. `orElse` returns the row or a default. `ifPresent` runs only if a row exists. `map` returns another Optional.

```
var user = new GlideQuery('sys_user')
  .where('email', 'abel.tuter@example.com')
  .selectOne('first_name')
  .orElse({ first_name: 'Nobody' });

var opt = new GlideQuery('sys_user')
  .selectOne('first_name');
if (opt.isPresent()) gs.info(opt.get().first_name);

.selectOne('first_name').where('active', true);
.selectOne('first_name').forEach(gs.info);
.selectOne('first_name').get(); // throws if empty
.orElse({ first_name: 'Nobody' }).isPresent();
```

`orElse` already unwrapped the Optional, so the result is a plain object. `insert`, `update`, and `insertOrUpdate` also return an Optional. `Stream.find` is the bridge: Stream in, Optional out.

Figure 3. Valid and invalid syntax once the builder has ended.

Stream

`map`, `filter`, `limit`, and `flatMap` return a Stream, so they chain, and they are lazy. `forEach`, `reduce`, and `toArray` end the Stream and run the query. `find` ends the Stream and returns an Optional, so Optional methods are legal after it. `toArray(n)` needs a cap. Prefer `GlideQuery.limit` before `select`. `Stream.limit` works, then `trims`, which reads more than you keep.

```
new GlideQuery('incident')
  .where('active', true)
  .select('number', 'short_description')
  .filter(function (row) { return row.short_description; })
  .map(function (row) { return row.number; })
  .forEach(gs.info);
```

```
var rows = new GlideQuery('incident').where('active', true).select('number').toArray(50);
```

```
new GlideQuery('task').select('description')
  .find(function (row) { return row.description; })
  .ifPresent(function (row) { gs.info(row.sys_id); });
```

Optional

`isPresent` and `isEmpty` return a boolean. `get` returns the row or throws "get() called on empty Optional" — use it only when a missing row is a bug. `orElse(default)` returns the row or the default, already unwrapped. `ifPresent(fn)` runs only when a row exists. `map(fn)` returns another Optional, so it can sit before `orElse` or `get`.

```

var user = new GlideQuery('sys_user')
  .where('email', 'abel.tuter@example.com')
  .selectOne('first_name', 'email')
  .orElse({ first_name: 'Nobody', email: '' });

var opt = new GlideQuery('sys_user').where('last_name', 'Luddy').selectOne('first_name');
if (opt.isPresent()) {
  gs.info(opt.get().first_name);
}

```

9. Writes

update needs a where on the primary key and returns an Optional of the updated row. updateMultiple takes a changes object and returns {rowCount}. There is no single-record delete. deleteMultiple deletes the rows matched by the preceding where clauses; del is an alias of deleteMultiple, not a second API. Docs describe both bulk writes as operating on the preceding where clauses. A bare updateMultiple or deleteMultiple with no where is unsupported. The exact error string is not quoted in the docs, so pair the same where chain with count before either call. Do not use deleteMultiple on tables with currency fields; currency child rows can be orphaned. Delete those one by one.

```

var n = new GlideQuery('incident').where('active', true).where('priority', 1).count();
gs.info('would update ' + n);

new GlideQuery('incident')
  .where('sys_id', '0123456789abcdef0123456789abcdef')
  .update({ short_description: 'corrected' });

new GlideQuery('incident')
  .where('active', false)
  .where('priority', 5)
  .updateMultiple({ urgency: 3 });

```

10. Names that cheat sheets disagree on

Use this	Not this	Note
orderBy / orderByDesc	orderByAsc	orderByAsc appears on some unofficial sheets. It is not in the official method list.
deleteMultiple	delete() / Del()	No single-record delete. del is the script-include alias of deleteMultiple.
where(field, 'IN', [...])	whereIn / whereNotIn	IN is an operator on where, not a separate documented method.
global.GlideQuery	GlideQuery in a scoped app	Scoped scripts must prefix the global scope.
GlideQuery.parse(table, eq)	query.parse(...)	parse is static.

Sources: ServiceNow GlideQuery API (Zurich developer reference and current scoped/global docs), the GlideQuery developer blog on data integrity and on there being no single-record delete, the Stream and Optional API pages, and the script-include alias of del to deleteMultiple. The method surface has changed across releases. If a name here is missing in a Background Script, inspect GlideQuery.prototype on that instance.